NODEBASE · RUNTIME INTERNALS

Workflow Execution: From Button Click to Execution Row

A complete trace of what happens after a user presses **Execute workflow**: the tRPC mutation, the Inngest event, the durable step function, the per-node executor loop, the Realtime channel fan-out that repaints the canvas, and the final write to the `Execution` table — including the failure path.

SOURCE OF TRUTH checked-out working tree DB EVIDENCE live Neon PostgreSQL RUNTIME Inngest + BullMQ + Redis

---

Scope and method

This document follows a single run of a workflow from the click in the editor to the terminal `SUCCESS` or `FAILED` row. It assumes the reader has seen *Visual Editor: Node Add, Client State & Save*, which covers how the graph is authored and persisted; everything from the moment execution is requested is treated as new.

Database claims come from live queries against Neon. Runtime claims — step checkpointing, retry counts, channel delivery — come from the installed Inngest SDK and the checked-in source. Where a claim could not be verified statically, it is labelled rather than asserted.

CONCERN	PRIMARY FILE
Trigger button	`src/features/editor/components/execute-workflow-button.tsx`
Dispatch mutation	`src/features/workflows/server/routers.ts`
Event send + dedupe	`src/inngest/utils.ts, src/lib/idempotency.ts`
Inngest client + serve route	`src/inngest/client.ts, src/app/api/inngest/route.ts`
Orchestrating function	`src/inngest/functions.ts`
Executor dispatch	`src/features/executions/lib/executor-registry.ts`
Per-node executors	`src/features/executions/components/*/executor.ts`
Realtime channels	`src/inngest/channels/*.ts`
Browser subscription	`src/features/executions/hooks/use-node-status.ts`
Run history UI	`src/features/executions/server/routers.ts`
Cron entry point	`src/queue/cron-worker.ts`

The run in one picture

Click ExecuteWorkflowButton → **tRPC execute** ownership check → **inngest.send** event queued → **executeWorkflow** step function → **Executor loop** topological order → **"Execution"** SUCCESS row

The HTTP response to the click returns as soon as the event is queued. Everything to the right of `inngest.send` happens out-of-band, which is why the editor learns about the run through a Realtime channel rather than through the mutation response.

PHASE 1 · THE BUTTON IS CONDITIONAL

The execute control is not always present. The editor renders it inside a bottom-centre `Panel` only when the graph already contains a manual trigger:

```
const hasManualTrigger = useMemo(() => {
  return nodes.some((node) => node.type === NodeType.MANUAL_TRIGGER);
}, [nodes]);
```

```
{hasManualTrigger && (
  <Panel position="bottom-center">
    <ExecuteWorkflowButton workflowId={workflowId} />
  </Panel>
)}
```

`editor.tsx:76-78, 103-107`

Because the predicate reads the live `nodes` array, the button appears and disappears as the user adds or deletes trigger nodes — it is derived state, not a stored permission. The component itself is deliberately thin: it holds no state, and its only job is to fire the mutation and let the hook own the pending flag.

```
const handleExecute = () => {
  executeWorkflow.mutate({ id: workflowId });
};

<Button size={"lg"} onClick={handleExecute} disabled={executeWorkflow.isPending}>
  <FlaskConicalIcon className="size-4" />
  Execute workflow
</Button>
```

`execute-workflow-button.tsx`

The `disabled={isPending}` guard is per-click only. Inngest enqueues on receipt, so the mutation resolves almost instantly and the button re-enables while the run is still executing. A second click during an in-flight run therefore creates a *second independent run* rather than being rejected — see Finding 1.

PHASE 2 · THE DISPATCH MUTATION

```
execute: protectedProcedure
  .input(z.object({ id: z.string() }))
  .mutation(async ({ input, ctx }) => {
    const workflow = await prisma.workflow.findUniqueOrThrow({
      where: { id: input.id, userId: ctx.auth.user.id },
```

```
    });

    await inngest.send({
      name: "workflows/execute.workflow",
      data: { workflowId: input.id },
    });

    return workflow;
  }),
```

`workflows/server/routers.ts`

Three things to note. The ownership check reuses the same `{ id, userId }` pattern as the editor's save path, so a user cannot execute somebody else's workflow. The event payload carries only `workflowId` — no `initialData`, because a manual run starts from an empty context. And the returned `workflow` object exists purely so the hook can read `workflow.name` for its success toast.

> **The manual path bypasses the shared dispatcher**
> The codebase has a helper, `sendWorkflowExecution`, that every other entry point uses. This mutation calls `inngest.send` directly instead. The helper does two things this skips: it attaches a **Redis idempotency key**, and it supplies a **client-generated event id** via `id: createId()`. Neither is needed for correctness of a single manual click, but the inconsistency means manual runs have no de-duplication guarantee while webhook runs do. Routing this through the helper — with no key, or a key of the execution id — would make the entry points uniform.

On the client, the mutation is wrapped with toasts only. There is no `invalidateQueries` for executions, because at click time no execution row exists yet; the list refreshes when the user navigates to it.

```
export const useExecuteWorkflow = () => {
  const trpc = useTRPC();
  return useMutation(
    trpc.workflows.execute.mutationOptions({
      onSuccess: (data) => { toast.success(`Workflow "${data.name}" executed`); },
      onError:  (error) => { toast.error(`Failed to executed workflow: ${error.message}`); },
    }),
  );
};
```

The `onError` string carries a grammatical slip — "Failed to executed workflow" — which is user-visible. Trivial to fix, noted only so it is not mistaken for a typo in this document.

PHASE 3 · THE EVENT AND ITS TRANSPORT

The Inngest client is a single module-level instance shared by the whole app:

```
export const inngest = new Inngest({
  id: "node4work",
  eventKey: process.env.INNGEST_EVENT_KEY,
  signingKey: process.env.INNGEST_SIGNING_KEY,
```

```
  middleware: [realtimeMiddleware()],
});
```

`src/inngest/client.ts`

Two details are load-bearing. `realtimeMiddleware()` is what gives the run function its `publish` capability, without which the whole status channel system in Phase 7 would not exist. And the app id `node4work` namespaces every event, so the function is addressed as `node4work:execute-workflow` in the Inngest dashboard.

The event name `workflows/execute.workflow` is a convention, not a framework requirement — the `/`-separated shape simply groups workflows under one filter in the dev UI. Exactly four places in the repository emit it:

EMITTER	ENTRY POINT	IDEMPOTENCY	INITIALDATA
workflows/server/routers.ts	Manual "Execute workflow" button	none	—
inngest/utils.ts	Shared helper (webhooks)	Redis `SET NX`	trigger-specific
queue/cron-worker.ts	BullMQ repeatable job	queue job id	`cron`
inngest/utils.ts	Telegram / Stripe / Google Form webhooks	per-source key	trigger-specific

Delivery is HTTP to a route that serves the function:

```
export const { GET, POST, PUT } = serve({
  client: inngest,
  functions: [executeWorkflow],
});
```

`src/app/api/inngest/route.ts`

Only one function is registered. In production Inngest Cloud reaches this route over an ngrok or public URL and retries the delivery; locally the Inngest dev server (`npm run inngest:dev`, port 8288) proxies to it. The `PUT` verb is what Inngest uses to *register* function metadata, and `GET` serves its health/inspect endpoint.

PHASE 4 · THE ORCHESTRATING FUNCTION

```
export const executeWorkflow = inngest.createFunction(
  {
    id: "execute-workflow",
    retries: process.env.NODE_ENV === "production" ? 3 : 0,
    onFailure: async ({ event, step }) => {
      return prisma.execution.update({
        where: { inngestEventId: event.data.event.id },
        data: {
          status: ExecutionStatus.FAILED,
          error: event.data.error.message,
          errorStack: event.data.error.stack,
        },
      });
```

```
    },
  },
  {
    event: "workflows/execute.workflow",
    channels: [ /* 14 channels, see Phase 7 */ ],
  },
  async ({ event, step, publish }) => { /* ... */ },
);
```

`src/inngest/functions.ts:22-57`

Three configuration decisions are worth pausing on.

- **Retry asymmetry.** Zero retries in development, three in production. Development gets fast, unambiguous failures; production absorbs transient upstream errors. The live data contains a run whose message is "*Failed after 3 attempts*", which confirms the production path is exercised.
- **Failure is persisted out-of-band.** `onFailure` is a separate Inngest-level hook, so the `FAILED` row is written even when the function body died in a way that skipped its own final step. It joins to the run by `event.data.event.id`.
- **Channel subscriptions live here.** All fourteen channels are declared on the function, which is what authorises `publish` against them. A channel not listed here cannot be published to, so adding a node type requires editing this array — one of the five touchpoints a new node type needs.

The body opens with a cheap guard, using `NonRetriableError` so a malformed event does not consume the retry budget:

```
const inngestEventId = event.id;
const workflowId = event.data.workflowId;

if (!inngestEventId || !workflowId) {
  throw new NonRetriableError("Workflow ID or Event ID is missing");
}
```

`functions.ts:59-64`

Why `step.run` matters

Every long-running operation in this function is wrapped in `step.run("name", fn)`. Inngest durably records the *result* of each completed step, so a retry replays the function body but returns memoised values for steps that already succeeded. This is what makes the three production retries safe: the `create-execution` step is not re-run (which would hit the unique index on `inngestEventId`), and a partially-completed pipeline does not re-fire the side effects of its earlier nodes. Per-node executors nest their own steps inside this framework, so the guarantee extends to individual node work.

`PHASE 5 · STEP ONE: CREATE THE EXECUTION ROW`

```
await step.run("create-execution", async () => {
  try {
```

```
    return prisma.execution.create({
      data: { workflowId, inngestEventId },
    });
  } catch (err) {
    throw new NonRetriableError(
      `create-execution failed: ${(err as Error).message}`,
    );
  }
});
```

`functions.ts:66-79`

The insert is minimal on purpose. `status` falls back to the schema default `RUNNING`, and `startedAt` to `now()`, so a row exists in a truthful state before any work begins. `inngestEventId` is the join key that every later write — the final success update and the `onFailure` hook — uses to find the row.

The `catch` converts any Prisma error into a `NonRetriableError`. Without it, a transient database blip would burn all three production retries against an operation that is deterministic, and the run would end with no `Execution` row at all. The trade-off is that a genuinely transient failure is recorded nowhere; the trade is deliberate — a run you can see is worth more than a rare retry.

> **Ordering consequence**
> The row is created *before* the graph is read. A run therefore always leaves a record, even when the workflow is fundamentally invalid. Two live `FAILED` rows carry the message "*Workflow contains a cycle*", which is exactly this property in action: the run was recorded, then the graph was found to be unsortable. This is a sound audit trail, and it is also why a cyclic workflow produces a durable error rather than a silent no-op.

`PHASE 6 · STEPS TWO AND THREE: RESOLVE THE PLAN`

Load and order the graph

```
const sortedNodes = await step.run("prepare-workflow", async () => {
  const workflow = await prisma.workflow.findUniqueOrThrow({
    where: { id: workflowId },
    include: { nodes: true, connections: true },
  });

  return topologicalSort(workflow.nodes, workflow.connections);
});
```

`functions.ts:81-93`

Note the deliberate omission of `userId` from the `where` clause, in contrast to every user-facing procedure. This is correct: the function is system-driven and the `workflowId` in the event was already authorised upstream (or originated from a signed webhook). The scoping that matters happens at the entry point, not here.

`topologicalSort` is covered in detail in the editor document. In execution context, two properties matter: isolated nodes receive a self-loop so they are not silently dropped, and a cycle is converted into the domain

error "*Workflow contains a cycle*" before any node runs.

That conversion is a *message match* against a third-party library, which is worth stating plainly because it is fragile. Sorting is delegated to the `toposort` package, which signals a cycle by throwing an `Error` whose message contains the substring `"Cyclic"`. The wrapper then pattern-matches on it:

```
try {
  sortedNodeIds = toposort(edges);
  sortedNodeIds = [...new Set(sortedNodeIds)];
} catch (error) {
  if (error instanceof Error && error.message.includes("Cyclic")) {
    throw new Error("Workflow contains a cycle");
  }
  throw error;
}
```

Any other error is rethrown untouched, so a genuine infrastructure failure is never mislabelled as a cycle — the discrimination is one-sided and safe. The residual risk is upstream: if `toposort` ever changes its error text, this branch stops matching and the raw library error reaches the user instead. Matching on a named error class, or pre-checking acyclicity with a graph walk, would remove the string coupling.

Resolve the owner

```
const userId = await step.run("find-user-id", async () => {
  const workflow = await prisma.workflow.findUniqueOrThrow({
    where: { id: workflowId },
    select: { userId: true },
  });

  return workflow.userId;
});
```

`functions.ts:95-104`

This is a second query against a row the previous step already fetched. `userId` could have been selected alongside the nodes for the price of one column. The cost is one extra round-trip per run — negligible against a median run of tens of seconds — and the benefit is that a change to what `prepare-workflow` returns cannot accidentally leak the owner into the serialised plan. It also keeps the sorted-node array free of fields the executor loop does not need.

`userId` exists for one reason: executors resolve the workflow owner's encrypted credentials. It is threaded into every executor call so that decryption uses the right key material without any executor needing to query the database itself.

PHASE 7 · THE EXECUTOR LOOP

Execution proper is a plain sequential loop:

```
let context = event.data.initialData || {};

for (const node of sortedNodes) {
  const executor = getExecutor(node.type as NodeType);

  context = await executor({
    data: (node.data as any) || {},
    nodeId: node.id,
    userId,
    context,
    step,
    publish,
  });
}
```

`functions.ts:106-121`

The whole data-flow model of the product sits in these ten lines. `context` is a single mutable object threaded through the pipeline: each executor receives everything accumulated so far and returns the new state. There are no per-node outputs, no variable scoping, and no branch semantics. The `await` in the loop is what serialises execution — nodes run strictly one at a time in topological order, so a node can safely read any key written by an earlier node.

`event.data.initialData` seeds that context for non-manual triggers: `{ googleForm: {...} }` for a form submission, `{ cron: {...} }` for a scheduled run, and so on. A manual run leaves it undefined, so the context starts as `{}`.

Dispatch by type

`getExecutor` is a typed registry lookup covering all fifteen `NodeType` variants. A missing entry throws `"No executor found for type: X"`, which would fail the run rather than skip a node silently — the right call for an orchestrator.

```
export const getExecutor = (type: NodeType): NodeExecutor<any> => {
  const executor = executorRegistry[type as keyof ExecutorRegistry];
  if (!executor) {
    throw new Error(`No executor found for type: ${type}`);
  }
  return executor as NodeExecutor<any>;
};
```

`executor-registry.ts`

> **INITIAL is aliased to the manual trigger**
> The registry maps `[NodeType.INITIAL]: manualTriggerExecutor`. This is the mitigation for the legacy `INITIAL` rows discussed in the editor document: those nodes are not dead weight at runtime, they execute as a pass-through manual trigger, so a workflow that still contains one runs rather than failing on an unmapped type. It also means a cyclic legacy workflow fails on the sort, not on the dispatch.

The executor contract

```
export interface NodeExecutorParams<TData = Record<string, unknown>> {
  data: TData;
  nodeId: string;
  userId: string;
  context: WorkflowContext;
  step: StepTools;
  publish: Realtime.PublishFn;
}

export type NodeExecutor<TData = Record<string, unknown>> = (
  params: NodeExecutorParams<TData>,
) => Promise<WorkflowContext>;
```

`src/features/executions/types.ts`

Every executor is handed the durable `step` tools, the `publish` function, and the running context — and must return the new context. Taking responsibility for both checkpointing and status reporting keeps the framework ignorant of node semantics.

Anatomy of a typical executor

Triggers are pure pass-throughs, which makes them the clearest baseline. Both `manualTriggerExecutor` and `cronTriggerExecutor` are the same three-beat shape:

```
await publish(manualTriggerChannel().status({ nodeId, status: "loading" }));

const result = await step.run("manual-trigger", async () => context);

await publish(manualTriggerChannel().status({ nodeId, status: "success" }));

return result;
```

`manual-trigger/executor.ts`

A working node adds real work, templating, and an error path. The HTTP executor is representative: it publishes `loading`, validates its configuration inside the step (publishing `error` and throwing `NonRetriableError` for each missing field), compiles the endpoint and body with Handlebars against the incoming context, performs the call, and folds the response into the context under the node's configured `variableName`:

```
const endpoint = Handlebars.compile(data.endpoint)(context);

const options: KyOptions = { method };

if (["POST", "PUT", "PATCH"].includes(method)) {
  const resolved = Handlebars.compile(data.body || "{}")(context);
  JSON.parse(resolved);
  options.body = resolved;
  options.headers = { "Content-Type": "application/json" };
}

const response = await ky(endpoint, options);
const responseData = contentType?.includes("application/json")
  ? await response.json()
```

```
  : await response.text();

return {
  ...context,
  [data.variableName]: { httpResponse: { status, statusText, data: responseData } },
};
```

`http-request/executor.ts`

That last spread is the mechanism by which output becomes input. The variable name is chosen by the user in the dialog, which is why the live `Execution.output` documents are keyed by arbitrary names such as `poki`, `gopi` and `myGemini` rather than by node type — the keys *are* the user's variable names. A later node reaches the value with a Handlebars expression like `{{gopi.httpResponse.data}}`.

The outer `try/catch` is what guarantees a status is always published: `loading` at entry, then either `success` or a catch-all `error` before the error is rethrown to fail the run. The consistent convention across all fifteen executors is what makes the shared `useNodeStatus` hook possible at all.

`PHASE 8 · INNGEST REALTIME CHANNELS`

Realtime is a one-way status feed from the run to the browser. It carries no workflow data — only `{ nodeId, status }` — and its sole purpose is to repaint node cards during an execution the browser did not start synchronously.

Channel definition

There is one channel module per node type, all structurally identical:

```
import { channel, topic } from "@inngest/realtime";

export const HTTP_REQUEST_CHANNEL_NAME = "http-request-execution";

export const httpRequestChannel = channel(HTTP_REQUEST_CHANNEL_NAME).addTopic(
  topic("status").type<{
    nodeId: string;
    status: "loading" | "success" | "error";
  }>(),
);
```

`src/inngest/channels/http-request.ts`

The typed `topic()` is a compile-time guarantee: a `publish` call with a malformed payload is a TypeScript error, not a message the browser silently discards. The wire contract is fixed across all fourteen channels — one topic named `status`, payload `{ nodeId, status: "loading" | "success" | "error" }`. The `nodeId` is essential because a channel is shared by every node of that type across every workflow, so the payload is the only thing that lets one card distinguish its own run from another user's.

CHANNEL	NODE TYPE
http-request-execution	HTTP_REQUEST
manual-trigger-execution	MANUAL_TRIGGER
cron-trigger-execution	CRON_TRIGGER
google-form-trigger-execution	GOOGLE_FORM_TRIGGER
stripe-trigger-execution	STRIPE_TRIGGER
gemini-execution	GEMINI
openai-execution	OPENAI
anthropic-execution	ANTHROPIC
discord-execution	DISCORD
slack-execution	SLACK
telegram-trigger-execution	TELEGRAM_TRIGGER
telegram-send-message-execution	TELEGRAM_SEND_MESSAGE
telegram-send-buttons-execution	TELEGRAM_SEND_BUTTONS
google-sheets-execution	GOOGLE_SHEETS

Fifteen channels exist; fourteen are subscribed on the function. `INITIAL` has no channel because it aliases the manual trigger's executor and its card reuses `manualTriggerChannel`.

Subscription token

The browser cannot subscribe directly; it needs a short-lived, scoped token minted server-side. Each node type has its own `"use server"` action for this:

```
export async function fetchHttpRequestRealtimeToken(): Promise<HttpRequestToken> {
  const token = await getSubscriptionToken(inngest, {
    channel: httpRequestChannel(),
    topics: ["status"],
  });

  return token;
}
```

`http-request/actions.ts`

The token is scoped to exactly one channel and one topic, so a compromised token cannot read another channel or another topic. Fourteen near-identical action files exist for this single reason; they are mechanical duplication that a generic helper keyed on the channel factory would collapse.

The browser side

```
const [status, setStatus] = useState<NodeStatus>("initial");

const { data } = useInngestSubscription({ refreshToken, enabled: true });
```

```
useEffect(() => {
  if (!data?.length) return;

  const latestMessage = data
    .filter((msg) =>
      msg.kind === "data" &&
      msg.channel === channel &&
      msg.topic === topic &&
      msg.data.nodeId === nodeId,
    )
    .sort((a, b) => new Date(b.createdAt).getTime() - new Date(a.createdAt).getTime())[0];

  if (latestMessage?.kind === "data") {
    setStatus(latestMessage.data.status as NodeStatus);
  }
}, [data, nodeId, channel, topic]);
```

use-node-status.ts

Each node card holds its own subscription and re-derives the full status list on every message. The filter narrows to its own `nodeId`, sorts newest-first, and takes the head — so out-of-order delivery resolves correctly, and a late-arriving `loading` cannot overwrite a newer `success`.

> **One subscription per node card**
> `enabled: true` is hard-coded, so a workflow with *n* nodes opens *n* subscriptions, each of which receives every message on its channel and discards all but its own. A ten-node workflow therefore holds ten tokens and ten message buffers, and a busy HTTP channel re-runs ten identical filter-and-sort passes per message. The hook does not take a `workflowId` or `runId`, so it also cannot distinguish two concurrent runs of the same workflow — a second run's `loading` will repaint a node that already succeeded in the first. Worth noting before adding a trigger-heavy workflow.

PHASE 9 · THE SUCCESS WRITE

```
await step.run("update-execution", async () => {
  return prisma.execution.update({
    where: { inngestEventId, workflowId },
    data: {
      status: ExecutionStatus.SUCCESS,
      completedAt: new Date(),
      output: context,
    },
  });
});

return { workflowId, result: context };
```

functions.ts:123-140

One update closes the run, and it does three things at once: flips the status, stamps `completedAt` so the run becomes measurable, and freezes the **entire final context** into the `output` jsonb column. The persisted

`output` is therefore a complete snapshot of everything the workflow produced — every intermediate node's contribution included, not just the terminal value.

The function's own return value is the same context, but it goes to Inngest, not to the user; nothing reads it back except the dev UI's run inspector.

Note the asymmetry in the `where` clause: the success path filters on both `inngestEventId` and `workflowId`, while `onFailure` filters on `inngestEventId` alone. Both work, since the event id is unique, but the stricter form is preferable and costs nothing.

PHASE 10 · THE FAILURE PATH

Two mechanisms can mark a run failed, and they are not alternatives — both can fire for one run.

An executor that throws propagates out of the `for` loop, so the `update-execution` step never runs. In production Inngest then retries the whole function up to three times, replaying memoised step results so completed nodes are not repeated. When the retries are exhausted, the `onFailure` hook executes:

```
onFailure: async ({ event, step }) => {
  return prisma.execution.update({
    where: { inngestEventId: event.data.event.id },
    data: {
      status: ExecutionStatus.FAILED,
      error: event.data.error.message,
      errorStack: event.data.error.stack,
    },
  });
},
```

functions.ts:26-37

A failed run has no end timestamp

`onFailure` writes `status`, `error` and `errorStack` but never `completedAt`. Confirmed against the live table: all 23 `FAILED` rows have `completedAt = NULL`, while all 43 `SUCCESS` rows have it set. Two consequences follow. Duration is uncomputable for failures, so any "slowest run" or timeout report silently excludes a third of the history. And a query such as "runs that have not finished" — `WHERE status = 'RUNNING'` — is the only remaining discriminator, which is a proxy for "never updated" rather than a real check. Adding `completedAt: new Date()` to the `onFailure` payload would close the gap, and the column is already nullable precisely so this write can be added.

Live failure evidence

The distinct error messages currently recorded, with counts:

```
// SELECT left("error",95), count(*) FROM "Execution" WHERE status='FAILED'
6  Bad Request: chat_id is empty
3  Invalid initialization vector
3  Request failed with status code 403 Forbidden: GET https://fakestoreapi.com/products/5
2  The caller does not have permission
```

```
2  Workflow contains a cycle
1  Failed after 3 attempts. Last error: You exceeded your current quota, please check your plan...
```

Each category maps to a distinct layer, and together they show the pipeline failing at every depth it can. *chat_id is empty* is node-level configuration reaching a real API. *Invalid initialization vector* is credential decryption — a Cryptr failure meaning the stored ciphertext no longer matches the `ENCRYPTION_KEY`, which is a deployment concern rather than a workflow defect. The 403 and permission errors are genuine upstream refusals. *Workflow contains a cycle* is a graph defect that never leaves the sort step. And *Failed after 3 attempts* is Inngest's retry exhaustion wrapper, confirming the production retry budget is live.

> **A cycle in a saved workflow is not hypothetical**
> The editor document noted that no cycle exists in the current 34 workflows, which remains true. But two `FAILED` executions carry "*Workflow contains a cycle*", so a cyclic graph was saved and executed at some point — either since repaired or the workflow since deleted. This is a real occurrence, not a theoretical one, and it confirms the editor accepts a configuration that the runtime cannot execute.

PHASE 11 · ALTERNATIVE ENTRY POINTS

Everything after `inngest.send` is identical for every trigger. Only the event payload and the de-duplication strategy differ, which is a good sign for the architecture — there is exactly one execution path.

Webhook triggers

The Google Form route shows the canonical shape: validate a query parameter, rate-limit, normalise the payload, and dispatch with a source-derived idempotency key.

```
const result = await sendWorkflowExecution({
  workflowId,
  idempotencyKey: `google-form:${body.responseId}`,
  initialData: { googleForm: formData },
});

if (!result) {
  return NextResponse.json({ success: true });
}
```

`src/app/api/webhooks/google-form/route.ts`

Deriving the key from a provider-side identifier is what makes a retried webhook safe: the same form response produces the same key, and the Redis `SET NX` lets only the first through.

```
export const acquireIdempotencyKey = async (
  key: string,
  ttlSeconds: number = DEFAULT_TTL,   // 86400 = 24h
): Promise<boolean> => {
  if (!redis) return true;            // fail-open when Upstash is unconfigured

  const result = await redis.set(key, "1", { nx: true, ex: ttlSeconds });
```

```
  return result === "OK";
};
```

src/lib/idempotency.ts

Note the fail-open branch: with no Upstash credentials the guard returns `true` and de-duplication is silently disabled rather than blocking execution. Defensible for a self-hosted setup, but it means the same endpoint has different delivery semantics in development and production. The 24-hour TTL is also short relative to the Stripe webhook retry window.

The equivalent routes are `/api/webhooks/stripe`, `/api/telegram/webhook/[workflowId]`, and `/api/webhooks/google-form`. Each normalises its provider payload into a namespaced `initialData` key so downstream nodes can address it unambiguously.

Cron triggers

Scheduled runs originate in BullMQ, not in a webhook. Saving a workflow calls `syncCronSchedules`, which reconciles repeatable jobs; when one fires, a separate worker process converts it into an Inngest event:

```
const worker = new Worker<CronJobData>("cron", async (job) => {
  const { workflowId, nodeId, scheduleId } = job.data;

  await inngest.send({
    name: "workflows/execute.workflow",
    data: {
      workflowId,
      initialData: {
        cron: { nodeId, scheduleId, timezone: job.data.timezone,
                timestamp: new Date().toISOString() },
      },
    },
  });
}, { connection, concurrency: 5 });
```

src/queue/cron-worker.ts

This is the one entry point that is neither user-authorised nor idempotency-guarded by `acquireIdempotencyKey`; de-duplication is inherited from BullMQ's job scheduler instead. The worker also refuses to start without an ioredis-compatible Upstash URL, exiting with a diagnostic rather than failing silently — a deliberate choice that turns a missing variable into an obvious boot failure. Because this path is reached from a queue, a user who has not saved a workflow recently sees no runs even if their cron trigger looks correct in the editor.

PHASE 12 · THE EXECUTION TABLE (VERIFIED AGAINST LIVE NEON)

Column types and nullability read from `information_schema.columns`.

COLUMN	SQL TYPE	NULL	ROLE
`id`	text	NO	cuid primary key
`workflowId`	text	NO	→ `"Workflow"(id)`, **ON DELETE CASCADE**
`status`	USER-DEFINED	NO	enum `RUNNING | SUCCESS | FAILED`, defaults `RUNNING`
`error`	text	YES	Message; set only by `onFailure`
`errorStack`	text	YES	Stack; set only by `onFailure`
`startedAt`	timestamp	NO	Default `now()` — the *only* time column that always populates
`completedAt`	timestamp	YES	Set on success only; **NULL for every failure**
`inngestEventId`	text	NO	**UNIQUE** — the join key for both terminal writes
`output`	jsonb	YES	Final `context` snapshot, frozen at completion

Physical table `"Execution"` · constraint `Execution_workflowId_fkey` · indexes `Execution_pkey` and `Execution_inngestEventId_key` (both **UNIQUE**)

> **There is no `createdAt`**
>
> Unlike `Workflow` and `Node`, this model has no `createdAt` — only `startedAt`. Every ordering and filtering query in the codebase uses it accordingly (`orderBy: { startedAt: "desc" }`). This is consistent, just worth stating explicitly, because it is the kind of column assumed present by reflex. It also means an `Execution` has no independent creation instant distinct from its start.

On the unique index: Prisma implements `@unique` as a unique *index* rather than a table constraint, so it does not appear in `pg_constraint`. An initial check of that catalogue appears to show the column is unindexed — it is not. `pg_indexes` confirms the btree index exists, and there are zero duplicate `inngestEventId` values across all rows. This matters: that index is what makes step memoisation safe, since a replayed `create-execution` step would otherwise insert a second row.

Live run history

```
// aggregate by status, with completion/error/output coverage
status   rows completedAt error output
SUCCESS     43          43         0       43
FAILED      23          0         23      0

// duration across completed runs
avg  33.39 s
max  222 s

// referential integrity
executions whose workflow is missing  0
duplicate inngestEventId groups       0
```

The coverage columns are the whole story in one line: `completedAt` and `error` are perfectly complementary, 43/0 on success and 0/23 on failure. No row has both, and none has neither — so the terminal state is unambiguous even though the failure timestamp is missing.

Success rate is 65%. The median run is tens of seconds, which is dominated by AI-model and third-party API latency rather than by orchestration; a 222-second maximum is consistent with retries and a cold Gemini call. These numbers also make the idempotency `NX` window meaningful: a 24-hour TTL is far longer than any single run.

Top-level keys across successful `output` documents, which are the user-chosen variable names from HTTP and action nodes:

```
poki 19x    telegram 12x    myGemini 11x
gopi  8x    feedback  6x    menu      6x
final  6x    geminiPoki 6x
```

Because `output` is the *cumulative* context rather than a terminal value, a single document routinely contains keys contributed by several nodes. This is the observable consequence of the threading model in Phase 7, and it is why the column is a full snapshot.

PHASE 13 · READING RUNS BACK

Two procedures serve the run history UI. Both scope by *relation filter* rather than by joining a user id onto `Execution` directly — `Execution` has no `userId` column, so ownership is reached through its parent workflow:

```
where: { workflow: { userId: ctx.auth.user.id } }
```

`src/features/executions/server/routers.ts`

That is the correct relational design, and it has a useful side effect: deleting a workflow cascades its entire execution history away, which is consistent with the same cascade that removes its nodes and connections.

`getMany` returns a paginated envelope (`items` , `totalCount` , `totalPages` , `hasNextPage` , `hasPreviousPage`) with the owning workflow's id and name denormalised into each row, ordered by `startedAt` descending. `getOne` resolves a single run the same way. The routes are `/executions` and `/executions/[executionId]` .

Note that a user sees only the runs of workflows they own — there is no shared-visibility model, so a run started by a webhook is visible to exactly the same audience as one started by a button.

Findings

1 — manual runs are not idempotency-guarded

`workflows.execute` calls `inngest.send` directly rather than the shared `sendWorkflowExecution` helper (`workflows/server/routers.ts`), so it acquires no Redis key and generates no event id. Every other entry point is guarded. Combined with a Save button that re-enables as soon as the mutation resolves rather than when the run finishes, a user double-clicking "Execute workflow" produces two independent runs and two `Execution` rows, both executing the full pipeline — so any node with an external side effect fires twice. This is the most user-visible risk in the runtime path, and the live history contains `output` documents with repeated variable names consistent with back-to-back runs. Routing the mutation through the helper is a one-line change.

2 — failed runs have no `completedAt`

`onFailure` omits it, so all 23 live `FAILED` rows have `completedAt = NULL` and failure duration is uncomputable. See Phase 10. One added field restores it.

3 — an OpenAI executor publishes to the Anthropic channel

`openai/executor.ts:52`

In the "Credential is missing" validation branch, the executor publishes via `anthropicChannel().status(...)` where every other publish in the same file uses `openAiChannel()` — a copy-paste error in a file that otherwise imports and uses both channels correctly. The consequence is precise: when an OpenAI node is missing its credential, the `error` status is published on the Anthropic channel, and since the OpenAI node card subscribes only to `openAi-execution`, **that node never turns red**. The run still fails and the error is still recorded on the `Execution` row, so this is a UI-only defect — but it is silent, and it hides the exact failure a user is most likely to hit while building a node. `openAiChannel` is already imported at line 7; line 52 is the only incorrect call.

4 — one Realtime subscription per node card, with no run scoping

`useNodeStatus` hard-codes `enabled: true` and filters only by `nodeId`. A ten-node workflow holds ten subscriptions that each receive and discard the same messages, and because nothing distinguishes run *A* from run *B*, concurrent executions of the same workflow paint over each other's statuses. A published payload carrying the Inngest run id — or a subscription keyed on the execution — would fix both the cost and the collision.

5 — cycle detection exists only at execution

The save path performs no acyclicity check, so a cyclic workflow persists cleanly and fails at run time with "Workflow contains a cycle" — which has happened twice in the live data. Because the `Execution` row is created before the sort, the failure is at least recorded durably, which is the right order. Validating in `workflows.update` would surface it at edit time instead.

6 — the idempotency guard fails open

`acquireIdempotencyKey` returns `true` when Upstash is unconfigured, silently disabling de-duplication rather than failing loudly. The same webhook therefore has different delivery semantics per environment. Failing open is defensible, but the 24-hour default TTL is also shorter than the Stripe webhook retry window.

7 — redundant `find-user-id` step

`prepare-workflow` already reads the full `Workflow` row, then a second step re-reads it for `userId`. One extra round-trip per run for a field that could ride along in the same `select`. Negligible next to a ~33-second median run, but it is a step Inngest must checkpoint and replay for no benefit.

8 — fourteen near-identical files for one pattern

Channel modules, realtime-token server actions, and the executors themselves all follow a fixed template differing only in the channel factory and the node-specific body. Collapsing the channel and token pairs into a single generic helper keyed on the channel factory would remove ~28 files and make the OpenAI/Anthropic mix-up in Finding 3 structurally impossible.

What is genuinely solid

The execution row is created before any work, so every run leaves an audit trail even when the workflow is unrunnable. Every long operation sits in a `step.run`, which is what makes the three production retries safe rather than destructive. The uniform `loading` → `success` / `error` publish convention across all fifteen executors is what allows a single shared hook to drive every node card. `NonRetriableError` is used correctly to distinguish deterministic failures from transient ones. And the entry-point design is clean: four different triggers, exactly one execution path, with trigger-specific data namespaced into `initialData` so the shared pipeline needs no knowledge of its callers.

Generated from the checked-out working tree and live database queries.